

Explorations with Backlight Controllers

Rohit Goswami · 2020-08-01 · tools · workflow

A short post detailing the many trials and tribulations of setting brightness on multiple monitors in tandem.

Background

As regular readers might know, I have a multi-screen setup, which accounts for having touch enabled on my primary laptop screen (detailed [here](#)). A failing of this setup was that I was not able to control the brightness of both monitors at the same time.

Existing Stack

Since I use `i3`, my brightness control is simply done with `bindsym` lines as follows^[^fn:1]:

```
bindsym XF86MonBrightnessDown exec light -U 10
bindsym XF86MonBrightnessUp exec light -A 10
```

Note that to get the right `bindsym` I use `screenkey` with the `keysyms` preference. My software of choice was Unfortunately, my existing setup (with `light`, since that is in the Arch `community` repo) did not actually allow dimming external screens arbitrarily. To be more exact,

```
light -h
```

Usage:

```
light [OPTIONS] [VALUE]
```

Commands:

```
-H, -h      Show this help and exit
-V          Show program version and exit
-L          List available devices
-A          Increase brightness by value
-U          Decrease brightness by value
-T          Multiply brightness by value (can be a non-whole number, ignores raw mode)
-S          Set brightness to value
-G          Get brightness
-N          Set minimum brightness to value
-P          Get minimum brightness
-O          Save the current brightness
-I          Restore the previously saved brightness
```

Options:

```
-r          Interpret input and output values in raw mode (ignored for -T)
-s          Specify device target path to use, use -L to list available
-v          Specify the verbosity level (default 0)
            0: Values only
            1: Values, Errors.
            2: Values, Errors, Warnings.
            3: Values, Errors, Warnings, Notices.
```

Copyright (C) 2012 - 2018 Fredrik Haikarainen

This is free software, see the source for copying conditions. There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE

Clearly it is possible to target specific devices, but for arbitrary additions this is quite tough. Additionally, the project has been more or less been stuck in [maintenance mode for a while now](#).

Multi-head Configurations

Exposing Brightness

The first hurdle faced in `o` was actually getting the external monitor to expose the right controls. This is accomplished neatly with `ddci-driver` [found here](#). As demonstrated (courtesy of [the ArchWiki](#)):

```
# Load module
sudo modprobe ddcci_backlight
# Check that it worked
sudo ddcutil capabilities | grep "Feature: 10"
sudo ddcutil getvcp 10
# Set brightness
sudo ddcutil setvcp 10 70
```

One of the obvious caveats of this technique is that `sudo` access or a dedicated `polkit` agent is required. My preferred method of loading this comes from [a comment](#) on the `ddcci-driver-linux-dkms` page of the AUR:

```
# Put in /etc/systemd/system/ddcci-backlight.service:
# https://aur.archlinux.org/packages/ddcci-driver-linux-dkms/
# Placing "ddcci_backlight" into /etc/modules-load.d
# leads to hang on boot with external (HDMI) monitor
# connected to the laptop, so we need to add the module later.

# And ddcci_backlight can't detect monitor during sddm.service startup.

[Unit]
After=multi-user.target
Before=sddm.service

[Service]
Type=oneshot
ExecStart=/usr/bin/modprobe ddcci_backlight
ExecStop=/usr/bin/modprobe --remove ddcci_backlight
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

This is then activated with a standard `systemctl enable ddcci-backlight.service` command. At this point, the device interface should be exposed to most backlight controllers.

Xrandr

This is the most obvious of all methods, and does not even require the `ddcci-driver`. We will simply tweak the brightness with `xrandr`.

```
# Get devices
xrandr | grep " connected"
# Tweak
xrandr --output DP1 --brightness 0.2
```

Note that this is an in-exact method, since it actually adjusts the gamma value instead, and it effectively tints your screen rather than modifying the brightness.

Clight

`clight` is an excellent, highly performant alternative to `redshift`, but it tends to force the main screen brightness to zero. Nothing which can't be configured away, but in practice, I work late and tend to turn off the tint anyway. This requires a daemon to be run, as well as needing to be turned on manually for `i3`. A very elegant additional feature gained by using `clight` is that external monitors are turned off automatically when lockscreens are activated.

Brillo

`brillo` is one of the newer controllers, and is pretty actively developed. The interface is almost exactly like `light`, and unlike `clight` there is no need to use a daemon. It meshes perfectly with a key-press based system like `i3` and also has controls for keyboard LEDs as well as for smoothed ramping up and down of the brightness. Most importantly, it features an `-e` flag which sets the brightness across all output devices. Essentially this means my configuration is simply modified to:

```
bindsym XF86MonBrightnessDown exec brillo -e -U 10
bindsym XF86MonBrightnessUp exec brillo -e -A 10
```

Conclusions

tl;dr moving from `light` to `brillo` with `ddcci-driver-linux-dkms` was a fantastic idea.